

ReconFormer: Evaluation of a Recurrent Transformer for Accelerated MRI Reconstruction

Muhammet Talat Yavuz - 22101763, Burcu Köprülü - 22103099,
Özgür Ekin Sönmez - 22201966, Eylül Naz Berçin - 22101937
EEE 475/575 – Medical Image Reconstruction and Processing
Bilkent University

Video Link: <https://www.youtube.com/watch?v=AZLNnjjzW9g>

Abstract—Magnetic resonance imaging (MRI) provides excellent soft-tissue contrast, but its acquisition is inherently slow because measurements are collected sequentially in k-space. A common acceleration strategy is to under-sample k-space, which reduces scan time but makes the reconstruction problem ill-posed and introduces aliasing artifacts in conventional zero-filled reconstructions. In this project, we study ReconFormer, a recurrent Transformer-based architecture for accelerated MRI reconstruction. ReconFormer combines recurrent pyramid Transformer layers with embedded data-consistency steps, allowing the network to exploit multi-scale image information while preserving consistency with the acquired k-space measurements. We evaluated the authors’ open-source PyTorch implementation using publicly available pre-trained weights on a subset of the fastMRI single-coil knee validation set. Reconstructions were generated at acceleration factors AF=4 and AF=8 and evaluated using PSNR, SSIM, and NMSE, together with qualitative comparisons against ground-truth and zero-filled images. Our results show that ReconFormer substantially suppresses the aliasing artifacts present in zero-filled reconstructions and produces images visually close to the fully sampled reference. Quantitatively, reconstruction quality decreases from AF=4 to AF=8, as expected from the more severe under-sampling, but the degradation trend is consistent with the original publication. These findings support ReconFormer’s effectiveness for accelerated MRI reconstruction while also highlighting limitations related to dataset dependence, sampling-pattern sensitivity, and reliance on pre-trained models.

I. INTRODUCTION

Magnetic resonance imaging (MRI) is a widely used medical imaging modality because it can directly visualize soft tissues and provide a wide range of contrast mechanisms. However, MRI acquisition is inherently slow. The scanner does not directly acquire an image in the spatial domain; instead, it collects samples in the frequency domain, commonly referred to as k-space. Since these samples are acquired over time, reducing the number of acquired k-space measurements is an important strategy for decreasing scan time.

Faster MRI acquisition is clinically important for several reasons. It can reduce patient discomfort, decrease motion-related artifacts, improve temporal resolution for dynamic imaging, and increase anatomical coverage within a fixed scan duration. A common way to accelerate MRI is to under-sample k-space. In Cartesian MRI, this often corresponds to skipping phase-encoding lines. However, when missing k-

space samples are simply set to zero and an inverse Fourier transform is applied, the resulting zero-filled reconstruction contains visible aliasing and blurring artifacts. Therefore, accelerated MRI reconstruction is an inverse problem: the goal is to recover a high-quality image from incomplete frequency-domain measurements.

Classical reconstruction methods address this ill-posed problem by introducing prior information about the image. Compressed sensing MRI, for example, exploits sparsity or compressibility of medical images in a suitable transform domain and solves an optimization problem that balances data fidelity with regularization [4]. More recently, deep learning-based methods have replaced hand-designed regularizers with learned image priors. CNN-based approaches, such as cascaded convolutional networks with data-consistency layers, can be interpreted as learned iterative reconstruction algorithms.

ReconFormer is a recent Transformer-based method proposed for accelerated MRI reconstruction [1]. Instead of relying only on local convolutional filters, ReconFormer uses recurrent Transformer blocks to capture long-range dependencies and multi-scale image features. Its architecture includes Recurrent Pyramid Transformer Layers (RPTLs), Recurrent Scale-wise Attention (RSA), and data-consistency layers. In this project, our objective was to study, run, and evaluate the official ReconFormer implementation rather than train a new model from scratch. We used the publicly available pre-trained checkpoints and the authors’ official code repository [2] to reconstruct fastMRI single-coil knee images at acceleration factors AF=4 and AF=8, evaluated the results using PSNR, SSIM, and NMSE, and compared them qualitatively against zero-filled reconstructions and fully sampled ground-truth images.

II. BACKGROUND AND METHOD

A. MRI Reconstruction as an Inverse Problem

In single-coil Cartesian MRI, the measured k-space data can be modeled as samples of the Fourier transform of the underlying image. Let $x \in \mathbb{C}^N$ denote the desired image, F the discrete Fourier transform operator, P an under-sampling mask that selects the acquired k-space locations, and y the measured k-space data. A simplified forward model is

$$y = PFx + n, \quad (1)$$

where n represents measurement noise. If k-space is fully sampled and the noise level is small, the image can be reconstructed by applying the inverse Fourier transform. In accelerated MRI, however, P selects only a subset of k-space. The resulting inverse problem is underdetermined because many images may be consistent with the acquired measurements.

The simplest reconstruction is zero-filled reconstruction. Missing k-space samples are set to zero, and the image is obtained by

$$x_{ZF} = F^{-1}P^T y, \quad (2)$$

where P^T inserts the acquired samples into their k-space locations and fills the missing entries with zeros. This reconstruction is fast but introduces aliasing artifacts because the missing frequency information is not recovered.

A more general reconstruction formulation is

$$\hat{x} = \arg \min_x \|PFx - y\|_2^2 + \lambda R(x), \quad (3)$$

where the first term enforces fidelity to the measured k-space samples, $R(x)$ is a regularization term encoding prior information about the image, and λ controls the balance between data fidelity and regularization. In classical methods, $R(x)$ may promote sparsity, smoothness, or other hand-designed image properties. In deep learning-based methods, this prior is learned from training data.

B. Data Consistency

Data consistency is central to many MRI reconstruction methods. The idea is that the reconstructed image should not contradict the k-space measurements that were actually acquired. If $\hat{x}$ is an intermediate reconstructed image, its Fourier transform $F\hat{x}$ can be corrected by replacing the predicted values at acquired k-space locations with the measured samples:

$$k_{DC}(i) = \begin{cases} y(i), & i \in \Omega, \\ F\hat{x}(i), & i \notin \Omega, \end{cases} \quad (4)$$

where Ω denotes the set of acquired k-space indices. The data-consistent image is then obtained as

$$x_{DC} = F^{-1}k_{DC}. \quad (5)$$

This operation is closely related to the data-fidelity term in Eq. (3). In learned reconstruction networks, data-consistency layers help prevent the model from producing reconstructions that look plausible but are inconsistent with the measurements.

C. CNN-Based Reconstruction

CNN-based MRI reconstruction methods often use a cascaded structure. A CNN block removes artifacts or refines the image estimate, and a data-consistency block enforces agreement with the acquired k-space samples. This structure resembles an unrolled iterative reconstruction algorithm, where the learned CNN acts as a data-driven regularizer.

In this report, we use D5C5 as the CNN baseline for contextual comparison. D5C5 is a deep cascade of convolutional neural networks with interleaved data-consistency steps [5]. In simple terms, the reconstruction is passed through a CNN block, corrected using data consistency, and then passed to the next cascade. This makes D5C5 conceptually close to classical iterative reconstruction methods, but with the regularization step learned from data.

D. ReconFormer Architecture

Fig. 1 illustrates the unrolled recurrent structure of ReconFormer. ReconFormer is designed to improve accelerated MRI reconstruction by replacing purely convolutional local modeling with recurrent Transformer-based global modeling [1]. Its main component is the Recurrent Pyramid Transformer Layer (RPTL), which processes image features at multiple spatial scales. Inside these layers, Recurrent Scale-wise Attention (RSA) allows the network to exchange information across scales and recurrent states.

The key motivation is that MRI reconstruction requires both local and global information. Local image details are needed to recover edges and fine anatomical structures, while global context helps resolve aliasing artifacts caused by under-sampling. CNNs can model local patterns effectively, but their receptive fields are limited unless the network becomes deep or explicitly multi-scale. Transformer attention provides a mechanism for capturing longer-range dependencies more directly.

ReconFormer also includes embedded data-consistency layers. Therefore, the method can be interpreted as a learned reconstruction pipeline that combines a Transformer-based image prior with explicit k-space measurement consistency. This connection makes ReconFormer closely related to the inverse-problem framework studied in class: the network learns the regularization component, while the data-consistency step preserves the measured Fourier samples.

III. DATA AND EXPERIMENTAL SETUP

A. Dataset

We evaluated ReconFormer on the fastMRI single-coil knee dataset [3]. fastMRI is a publicly available benchmark dataset for accelerated MR image reconstruction. In this project, we used the validation split and did not perform training. The evaluation subset contained 30 scans, corresponding to 1059 slices in total.

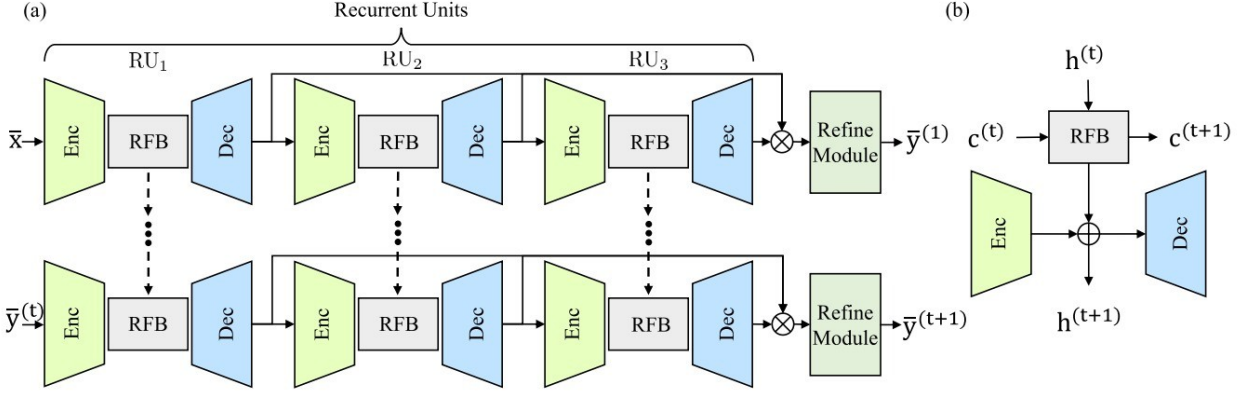


Fig. 1: Architecture of ReconFormer. The method is organized as an unrolled recurrent reconstruction pipeline consisting of recurrent units. Each unit contains encoder and decoder modules together with ReconFormer Blocks (RFBs), and the final output is refined by a refine module. The recurrent structure enables information propagation across reconstruction stages, while the RFBs provide the Transformer-based feature modeling used by the method. Figure adapted from [1].

The data used by the ReconFormer repository are preprocessed to match the expected input format of the authors’ data loader. Each scan is stored as an HDF5 file containing complex-valued k-space data and the corresponding fully sampled reconstruction used as the ground-truth reference. We evaluated two acceleration factors, AF=4 and AF=8, using the corresponding pre-trained checkpoints provided by the authors.

For AF=4, the center fraction was 0.08, and for AF=8, the center fraction was 0.04, following the settings used by the original implementation. These center fractions specify the fully sampled low-frequency k-space region retained in the under-sampling mask. The remaining k-space locations are under-sampled according to the acceleration setting.

B. Computational Environment

Experiments were run on Google Colab using an NVIDIA Tesla T4 GPU with 15 GB of memory. The original ReconFormer repository was written for an older software environment based on PyTorch 1.7 and Python 3.6. Since this environment is difficult to reproduce on current systems, we used the PyTorch version provided by Colab and made only the compatibility changes required to run inference.

The model architecture and learned weights were not modified. The project focused on testing the available pre-trained model and evaluating the reconstruction results, consistent with the undergraduate project objective of running and analyzing a recent open-source reconstruction method.

C. Implementation Modifications

Four implementation adjustments were required to run the evaluation and produce the required qualitative results. Only the first three were necessary for runtime compatibility; the fourth was added to export visual results for qualitative analysis. None of these changes modified the model architecture,

learned weights, data-consistency operation, or reconstruction algorithm.

First, the evaluation script was run in single-GPU mode by passing `-gpu 0`. This avoided the repository’s default multi-GPU behavior, which caused device selection errors on Colab.

Second, deprecated Fourier-transform functions were updated. The original code used older PyTorch functions such as `torch.fft`, `torch.ifft`, `torch.rfft`, and `torch.irfft`, which were removed in newer PyTorch versions. We rewrote the relevant Fourier-transform utilities using the modern `torch.fft` module. The original code represented complex numbers using real-valued tensors with a final dimension of size 2, corresponding to real and imaginary components. Our update preserved this input-output convention, so the rest of the repository remained compatible.

Third, the test batch size was reduced from 8 to 1 because the attention layers of ReconFormer exceeded the memory capacity of the T4 GPU at the original batch size. This change affects only runtime, not the reconstruction result for each slice, because inference is applied independently to each input.

Fourth, we extended the evaluation function to save qualitative comparison figures. The original `test_recon_save` function in `models/evaluation.py` computed PSNR, SSIM, and NMSE and internally stored reconstructed outputs, ground-truth targets, and zero-filled inputs, but it did not write images to disk. We added a visualization block before the return statement to save a four-panel figure showing the ground truth, zero-filled reconstruction, ReconFormer reconstruction, and absolute error map for a representative central slice. This addition does not affect the metric computation or the returned values; it only exports visual results for qualitative analysis.

IV. EVALUATION METHODS

We evaluated reconstruction quality using both quantitative and qualitative analysis.

A. Quantitative Metrics

The quantitative metrics were computed using the fully sampled reference images as ground truth. We used PSNR, SSIM, and NMSE.

The normalized mean squared error is defined as

$$\text{NMSE}(x, \hat{x}) = \frac{\|x - \hat{x}\|_2^2}{\|x\|_2^2}, \quad (6)$$

where x is the ground-truth image and $\hat{x}$ is the reconstructed image. Lower NMSE indicates better reconstruction.

The peak signal-to-noise ratio is

$$\text{PSNR}(x, \hat{x}) = 20 \log_{10} \left(\frac{x_{\max}}{\sqrt{\text{MSE}(x, \hat{x})}} \right), \quad (7)$$

where $x_{\max}$ is the maximum possible image intensity and MSE is the mean squared error. Higher PSNR indicates better pixel-wise fidelity.

SSIM measures structural similarity between two images. It compares luminance, contrast, and structural information and is given by

$$\text{SSIM}(x, \hat{x}) = \frac{(2\mu_x\mu_{\hat{x}} + C_1)(2\sigma_{x\hat{x}} + C_2)}{(\mu_x^2 + \mu_{\hat{x}}^2 + C_1)(\sigma_x^2 + \sigma_{\hat{x}}^2 + C_2)}, \quad (8)$$

where μ_x and $\mu_{\hat{x}}$ are local means, σ_x^2 and $\sigma_{\hat{x}}^2$ are local variances, $\sigma_{x\hat{x}}$ is the local covariance, and C_1, C_2 are constants for numerical stability. Higher SSIM indicates better structural similarity.

B. Qualitative Evaluation

For qualitative evaluation, we compared ground-truth images, zero-filled reconstructions, ReconFormer reconstructions, and error maps. Error maps were computed as the absolute difference between the ground-truth and reconstructed images:

$$e = |x - \hat{x}|. \quad (9)$$

This allows us to identify where the model makes residual reconstruction errors. We focused on whether aliasing artifacts were removed, whether tissue boundaries were preserved, and whether errors increased when the acceleration factor changed from AF=4 to AF=8.

For reproducibility, the qualitative figures were generated from a fixed representative case. The evaluation function selected the first scan after sorting by filename and used its central slice. Since the same scan and slice were used for both AF=4 and AF=8, the resulting figures allow a direct visual comparison between acceleration factors.

C. Baselines

The first baseline is the zero-filled reconstruction, which directly shows the artifacts caused by under-sampling before learned reconstruction. The second baseline is D5C5, a CNN-based cascaded reconstruction method [5]. We did not independently run D5C5. Instead, we used the D5C5 results reported in Table I of the ReconFormer paper under the same fastMRI setting [1]. This comparison is included to contextualize ReconFormer relative to a conventional CNN-based method.

V. RESULTS

A. Quantitative Results

Table I summarizes our ReconFormer evaluation results on 30 fastMRI single-coil knee validation scans. Metrics were computed per volume and averaged across scans.

TABLE I: ReconFormer results from our evaluation on 30 fastMRI single-coil knee validation scans.

Acceleration	PSNR (dB)	SSIM	NMSE
AF=4	31.69	0.797	0.0345
AF=8	29.88	0.738	0.0477
AF=4 → AF=8	−1.81	−0.059	+0.0132

As expected, reconstruction quality decreases when the acceleration factor increases from AF=4 to AF=8. The PSNR decreases by 1.81 dB, SSIM decreases by 0.059, and NMSE increases by 0.0132. This is physically meaningful because AF=8 corresponds to more aggressive k-space under-sampling, so fewer measurements are available for recovering the same image.

Table II compares our results with the ReconFormer values reported in the original paper on fastMRI. The paper reports results averaged over 199 scans, whereas our evaluation uses a subset of 30 scans. Therefore, exact equality is not expected. However, the degradation trend from AF=4 to AF=8 closely matches the paper: our PSNR drop is 1.81 dB, while the paper reports a 1.84 dB drop. This supports that our inference and evaluation pipeline behaves consistently with the original implementation. Although our SSIM values are higher than those reported in the paper while PSNR and NMSE are slightly worse, this difference can be attributed to the smaller 30-scan evaluation subset and possible implementation-level details in metric computation, such as image normalization, data range, or SSIM window settings.

TABLE II: Comparison between our ReconFormer results and the ReconFormer values reported in the original paper on fastMRI.

Source	AF	PSNR (dB)	SSIM	NMSE
Ours	4	31.69	0.797	0.0345
Ours	8	29.88	0.738	0.0477
Paper [1]	4	32.73	0.7383	0.0310
Paper [1]	8	30.89	0.6697	0.0429

Table III compares ReconFormer with D5C5 using values reported in the original ReconFormer paper. ReconFormer outperforms D5C5 at both acceleration factors while using fewer parameters. The PSNR advantage increases from 0.48 dB at AF=4 to 1.24 dB at AF=8. This suggests that the Transformer-based architecture becomes more beneficial when the reconstruction problem becomes more severely ill-posed.

TABLE III: CNN baseline comparison on fastMRI using values reported in the ReconFormer paper. D5C5 was not independently reproduced in this project.

Method	Param.	AF=4 PSNR	AF=4 SSIM	AF=8 PSNR	AF=8 SSIM
D5C5 [5]	2.237M	32.25	0.7256	29.65	0.6457
ReconFormer [1]	1.141M	32.73	0.7383	30.89	0.6697

B. Qualitative Results

Figures 2 and 3 show qualitative results for the same representative central knee slice at AF=4 and AF=8. Each figure includes the ground-truth image, zero-filled reconstruction, ReconFormer reconstruction, and the corresponding error map.

The zero-filled images demonstrate the direct effect of under-sampling. At AF=4, aliasing artifacts and blurring are visible, but the anatomy remains partially recognizable. At AF=8, the artifacts are more severe, and the joint structure becomes more difficult to identify in the zero-filled reconstruction. This agrees with the quantitative metrics, where AF=8 produces lower PSNR and SSIM and higher NMSE.

ReconFormer substantially improves the visual quality at both acceleration factors. The reconstructed images recover the main anatomical structures and remove most of the coherent artifacts visible in the zero-filled baseline. The error maps show that the remaining errors are not uniformly distributed. Instead, they are concentrated near edges, tissue boundaries, and high-frequency details. This is consistent with the k-space interpretation of MRI: low-frequency information captures coarse image contrast, while high-frequency information corresponds to fine details and edges, which are more difficult to recover after aggressive under-sampling.

VI. DISCUSSION

Our results show that ReconFormer can reconstruct highly under-sampled single-coil knee MR images with substantially improved visual quality compared with zero-filled reconstruction. The method is particularly effective at suppressing aliasing artifacts caused by missing k-space lines. Quantitatively,

the expected degradation from AF=4 to AF=8 is observed across all three metrics. This confirms that the reconstruction problem becomes more difficult as fewer k-space measurements are available.

ReconFormer is closely related to the inverse-problem framework covered in the course. The measured k-space data provide incomplete Fourier-domain observations of the object. Classical reconstruction methods combine a data-fidelity term with an explicit regularizer. ReconFormer follows the same conceptual structure, but the regularization is learned by a recurrent Transformer network. The data-consistency layers preserve the acquired k-space samples, corresponding to the measurement-fidelity component of the reconstruction objective.

The comparison with D5C5 illustrates the difference between convolutional and Transformer-based learned priors. Both methods use an iterative reconstruction structure with data-consistency enforcement. However, D5C5 uses convolutional blocks, whose kernels are local and mostly content-independent. ReconFormer uses scale-wise attention to capture multi-scale features and longer-range dependencies. According to the original paper’s reported fastMRI results, ReconFormer achieves higher PSNR and SSIM than D5C5 while using fewer parameters. The improvement is larger at AF=8, suggesting that global context modeling becomes more valuable as the inverse problem becomes more ill-posed.

There are several limitations to our evaluation. First, we used pre-trained weights and did not train the model from scratch. Therefore, the results depend on the training distribution and implementation choices of the original authors. Second, our evaluation used 30 validation scans rather than the full 199-scan fastMRI validation set used in the paper, so exact metric values are not expected to match. Third, we did not independently reproduce all CNN baselines; D5C5 values were taken from the original paper. Finally, our experiments focused on single-coil knee MRI and did not test robustness to other anatomies, multi-coil data, different noise levels, or out-of-distribution sampling masks.

Future work could extend the evaluation to the full fastMRI validation set, test additional sampling patterns, evaluate multi-coil reconstruction, and compare ReconFormer with more recent reconstruction models. It would also be important to examine failure cases carefully, since medical reconstruction algorithms must preserve diagnostically relevant details and should not hallucinate structures that are not supported by the acquired measurements.

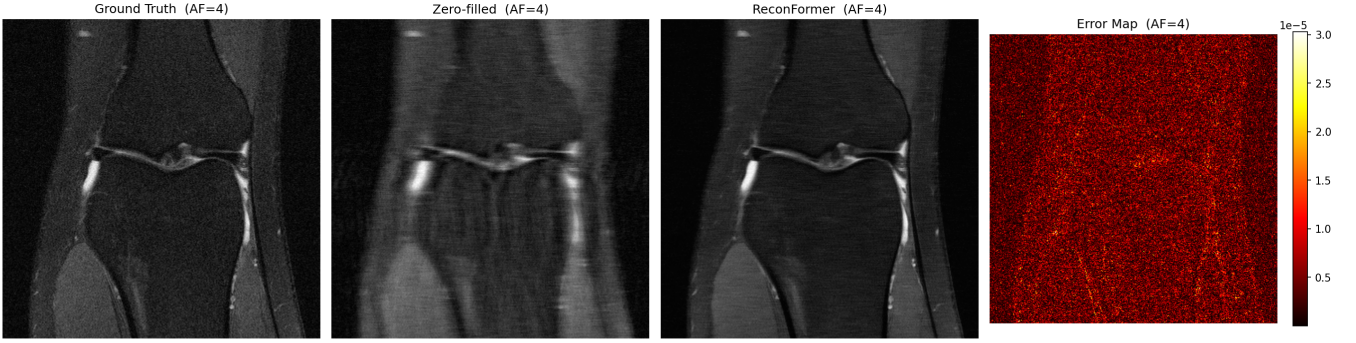


Fig. 2: Qualitative reconstruction results at $AF=4$. The zero-filled reconstruction contains visible under-sampling artifacts and blurring. ReconFormer suppresses these artifacts and produces a reconstruction visually close to the ground truth. Residual errors are mainly concentrated around tissue boundaries and high-frequency structures.

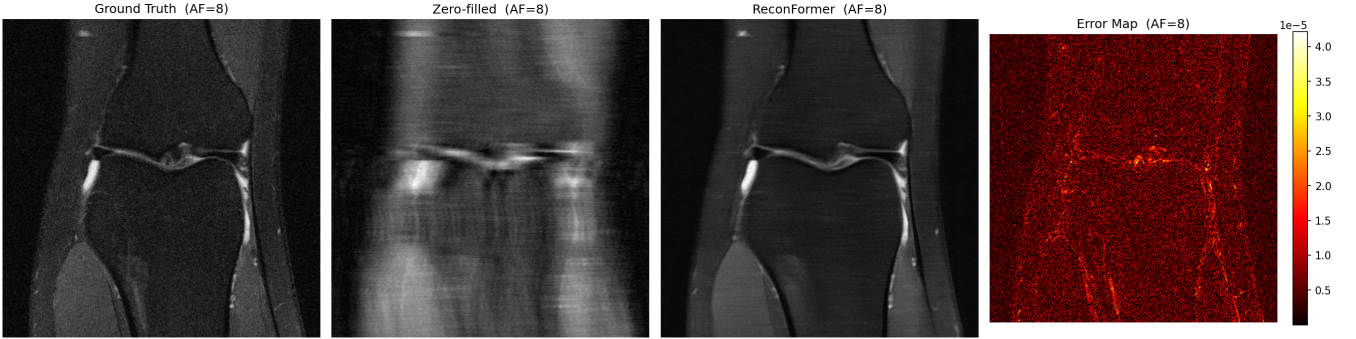


Fig. 3: Qualitative reconstruction results at $AF=8$. Compared with $AF=4$, the zero-filled reconstruction contains stronger artifacts because fewer k-space samples are acquired. ReconFormer still recovers the main anatomical structures, but the error map shows larger residual errors, especially near tissue boundaries and the joint region.

VII. INDIVIDUAL CONTRIBUTIONS

Muhammet Talat Yavuz contributed to running the ReconFormer implementation, debugging code compatibility issues, quantitative evaluation, and report preparation.

Burcu Köprülü contributed to method analysis, qualitative result interpretation, and report preparation.

Özgür Ekin Sönmez contributed to the literature review, interpretation of accelerated MRI reconstruction methods, and presentation preparation.

Eylül Naz Berçin contributed to literature review, figure preparation, qualitative analysis, and video preparation.

GENAI DISCLOSURE

Generative AI tools were used for brainstorming, debugging support, and improving the clarity of written explanations. In particular, AI assistance helped identify compatibility issues between the original ReconFormer code and modern PyTorch versions, suggested updates for deprecated Fourier-transform API calls, and supported writing the qualitative figure-generation block. The group reviewed, tested, and verified all code modifications and results. The group remains fully responsible for the submitted material.

APPENDIX A IMPLEMENTATION DETAILS

We used the official ReconFormer repository and the publicly available pre-trained checkpoints provided by the authors. The model was evaluated only in inference mode; no training or fine-tuning was performed.

The original code was written for an older PyTorch environment. To run the evaluation on Google Colab, deprecated Fourier-transform functions were updated using the modern `torch.fft` module. The original implementation represented complex-valued data as real tensors with a final dimension of size 2. The updated implementation preserved this representation by converting the tensor to a native complex tensor before applying the Fourier transform and converting it back afterward.

The evaluation script was also set to single-GPU mode, and the batch size was reduced from 8 to 1 to avoid GPU memory errors. These changes do not affect the mathematical reconstruction algorithm or the model architecture. They only make the original inference pipeline compatible with the available computational environment.

In addition, we extended the evaluation function to save qual-

itative figures. The figure-generation block reuses the already computed dictionaries `outputs_save`, `targets_save`, and `input_save`. It selects the first scan after sorting by filename and uses its central slice. Since the same scan and slice are used for AF=4 and AF=8, the qualitative figures allow a direct comparison between acceleration factors. This modification only writes visual outputs to disk and does not affect numerical metric computation.

APPENDIX B CODE AND REPRODUCIBILITY

The main evaluation script used was `main_recon_test.py` from the official ReconFormer repository. The relevant inputs were the fastMRI validation data path, the acceleration factor, the sequence type, and the corresponding pre-trained checkpoint. The evaluated acceleration factors were AF=4 and AF=8.

For AF=4, we used the corresponding AF=4 pre-trained checkpoint and center fraction 0.08. For AF=8, we used the corresponding AF=8 pre-trained checkpoint and center fraction 0.04. The qualitative figures were generated automatically during evaluation after adding the visualization block to `test_recon_save`.

The quantitative metrics were computed per scan volume and averaged over 30 validation scans, corresponding to 1059 slices in total.

APPENDIX C IMPLEMENTATION MODIFICATIONS

This appendix summarizes the main code-level modifications made to run the official ReconFormer implementation in the current Google Colab environment and to export qualitative results. These changes were made only for compatibility and evaluation purposes. They did not modify the ReconFormer model architecture, learned weights, data-consistency operation, or reconstruction algorithm.

A. Updated Fourier Transform Functions

The original ReconFormer code was written for an older PyTorch version and used deprecated Fourier-transform functions. In `data/transforms.py`, the functions `fft2`, `ifft2`, `rfft2`, and `irfft2` were updated to use the modern `torch.fft` module.

The original functions used PyTorch 1.x API calls such as `torch.fft`, `torch.ifft`, `torch.rfft`, and `torch.irfft`. These functions are no longer supported in recent PyTorch versions. Therefore, they were replaced with `torch.fft.fftn` and `torch.fft.ifftn`.

The original implementation represents complex-valued data using real-valued tensors with a final dimension of size 2, where the last dimension stores the real and imaginary components. The updated implementation preserves this convention by using `torch.view_as_complex` before applying the

Fourier transform and `torch.view_as_real` after the transform. The `normalized=True` argument in the old implementation was mapped to `norm='ortho'` in the new `torch.fft` API. The `.contiguous()` operation was also added because `torch.view_as_complex` requires a contiguous memory layout.

The updated Fourier-transform functions are shown below:

```
def fft2(data, normalized=True):
    assert data.size(-1) == 2
    data = ifftshift(data, dim=(-3, -2))
    norm = 'ortho' if normalized else 'backward'
    data = torch.view_as_real(
        torch.fft.fftn(
            torch.view_as_complex(data.contiguous()),
            dim=(-2, -1),
            norm=norm
        )
    )
    data = fftshift(data, dim=(-3, -2))
    return data

def rfft2(data):
    data = ifftshift(data, dim=(-2, -1))
    data = torch.view_as_real(
        torch.fft.fftn(
            data.contiguous(),
            dim=(-2, -1),
            norm='ortho'
        )
    )
    data = fftshift(data, dim=(-3, -2))
    return data

def ifft2(data, normalized=True):
    assert data.size(-1) == 2
    data = ifftshift(data, dim=(-3, -2))
    norm = 'ortho' if normalized else 'backward'
    data = torch.view_as_real(
        torch.fft.ifftn(
            torch.view_as_complex(data.contiguous()),
            dim=(-2, -1),
            norm=norm
        )
    )
    data = fftshift(data, dim=(-3, -2))
    return data

def irfft2(data):
    data = ifftshift(data, dim=(-3, -2))
    data = torch.fft.ifftn(
        torch.view_as_complex(data.contiguous()),
        dim=(-2, -1),
        norm='ortho'
    ).real
    data = fftshift(data, dim=(-2, -1))
    return data
```

B. Reduced Test Batch Size

The second implementation change was made in `main_recon_test.py`. The original testing script created the validation dataset with a batch size of 8. However, during inference on Google Colab with an NVIDIA Tesla T4 GPU, this batch size caused memory issues because ReconFormer’s attention layers require a large amount of GPU memory.

Therefore, the batch size argument in the `_create_dataset` call was reduced from 8 to 1. The original code was:

```

if args.phase == 'test':
    dataset_val = _create_dataset(
        path_dict[args.test_dataset]/args.sequence,
        val_data_transform,
        'val',
        args.sequence,
        8,
        False,
        1.0
    )

```

The modified code was:

```

if args.phase == 'test':
    dataset_val = _create_dataset(
        path_dict[args.test_dataset]/args.sequence,
        val_data_transform,
        'val',
        args.sequence,
        1,
        False,
        1.0
    )

```

Only the fifth positional argument, corresponding to the batch size, was changed. This modification affects only memory usage and runtime. Since inference is applied independently to each slice or volume, reducing the batch size does not change the reconstructed output or the computed metrics.

C. Added Qualitative Figure Generation

The final modification was added to `models/evaluation.py` inside the `test_recon_save` function. The original function computed and printed the quantitative metrics, but although it populated the dictionaries `outputs_save`, `targets_save`, and `input_save`, it did not save any visual results to disk.

To support qualitative analysis, a visualization block was inserted before the return statement of the function. This block selects the first scan after sorting by filename, takes its central slice, and saves a four-panel figure containing the ground-truth image, zero-filled reconstruction, ReconFormer reconstruction, and absolute error map.

The added block is shown below:

```

# ===== QUALITATIVE FIGURES =====
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import os as _os

af = args.accelerations[0] if isinstance(
    args.accelerations, list
) else args.accelerations

fig_dir = _os.path.join('/content', 'figures')
_os.makedirs(fig_dir, exist_ok=True)

sample_fname = sorted(outputs_save.keys())[0]
vol_out = outputs_save[sample_fname]
vol_tgt = targets_save[sample_fname]
vol_inp = input_save[sample_fname]
mid = vol_out.shape[0] // 2

gt = vol_tgt[mid]
zf = vol_inp[mid]
recon = vol_out[mid]
err = np.abs(gt - recon)

```

```

fig, axes = plt.subplots(1, 4, figsize=(18, 5))

for ax, img, title, cmap in zip(
    axes,
    [gt, zf, recon, err],
    ['Ground Truth', 'Zero-filled',
     'ReconFormer', 'Error Map'],
    ['gray', 'gray', 'gray', 'hot']):

    im = ax.imshow(img, cmap=cmap)

    if title == 'Error Map':
        plt.colorbar(im, ax=ax, fraction=0.046)

    ax.set_title(f'{title} (AF={af})', fontsize=12)
    ax.axis('off')

plt.tight_layout()

save_path = _os.path.join(
    fig_dir,
    f'qualitative_AF{af}.png'
)

plt.savefig(save_path, dpi=150, bbox_inches='tight')
plt.close(fig)

print(
    'Saved qualitative figure:',
    save_path,
    '| scan:',
    sample_fname,
    '| slice:',
    mid
)

# ===== END QUALITATIVE FIGURES =====

```

This block only exports visual outputs and does not alter the metric computation. The function's return statement remains unchanged:

```

return metrics['val_loss'], metrics['nmse'], metrics['ssim'], metrics['psnr']

```

The saved qualitative figures were used in the main report to compare the ground truth, zero-filled reconstruction, ReconFormer output, and residual error map at AF=4 and AF=8.

REFERENCES

- [1] P. Guo, Y. Mei, J. Zhou, S. Jiang, and V. M. Patel, "ReconFormer: Accelerated MRI Reconstruction Using Recurrent Transformer," *IEEE Transactions on Medical Imaging*, vol. 43, no. 1, pp. 582–593, Jan. 2024, doi: 10.1109/TMI.2023.3314747.
- [2] P. Guo, "ReconFormer official implementation," GitHub repository. [Online]. Available: <https://github.com/guopengf/ReconFormer>
- [3] F. Knoll *et al.*, "fastMRI: A Publicly Available Raw k-Space and DICOM Dataset of Knee Images for Accelerated MR Image Reconstruction Using Machine Learning," *Radiology: Artificial Intelligence*, vol. 2, no. 1, e190007, Jan. 2020.
- [4] M. Lustig, D. Donoho, and J. M. Pauly, "Sparse MRI: The application of compressed sensing for rapid MR imaging," *Magnetic Resonance in Medicine*, vol. 58, no. 6, pp. 1182–1195, Dec. 2007.
- [5] J. Schlemper, J. Caballero, J. V. Hajnal, A. N. Price, and D. Rueckert, "A Deep Cascade of Convolutional Neural Networks for Dynamic MR Image Reconstruction," *IEEE Transactions on Medical Imaging*, vol. 37, no. 2, pp. 491–503, Feb. 2018.